

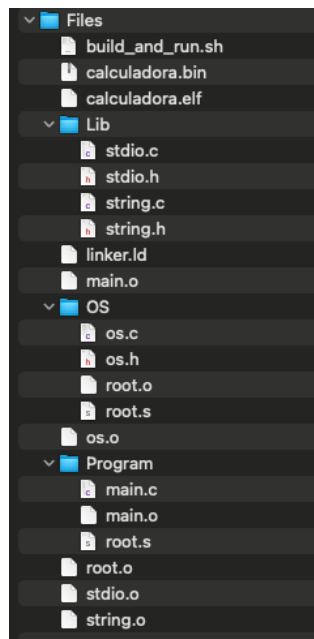
Lab 2

Overview de la arquitectura

Este laboratorio implementa un sistema pequeño organizado en tres capas. Cada una de estas capas tiene responsabilidades específicas y se comunican entre ellas. Esta separación hace que el sistema sea más fácil de entender y a la larga, extender.

Las capas son las siguientes:

1. Usuario
2. Librerías
3. Sistema Operativo



1. Capa de Usuario

Esta es la capa de más alto nivel, y contiene la lógica principal del programa, pero no nos dice cómo funciona todo a detalle, ni se comunica directamente con el hardware.

Sus responsabilidades son:

- Solicitar datos al usuario
- Realizar cálculos (sumar números)
- Mostrar resultados

La capa de usuario no interactúa directamente con el hardware. Utiliza funciones de la capa de librerías como PRINT y READ.

2. Capa de Librerías

Esta capa funciona como un intermediario entre el usuario y el sistema operativo. Traduce operaciones de alto nivel a operaciones de bajo nivel. Implementa `stdio.c` y `string.c`.

Funciones principales

- PRINT: Convierte números a texto y los envía carácter por carácter
- READ: Recibe texto y lo convierte al tipo de dato solicitado

Funciones de conversión

- `itoa()`: convierte un entero a un string
- `atoi()`: convierte un string a un entero
- `ftoa()`: convierte un float a un string
- `atof()`: convierte un string a un float

La librería no accede directamente a registros de hardware, solo llama a funciones del sistema operativo.

3. Sistema Operativo

Esta es la capa más baja y la única que interactúa directamente con el hardware.

Se encarga de:

- Manejar registros de memoria UART
- Enviar caracteres (`uart_putc`)
- Recibir caracteres (`uart_getc`)
- Enviar strings (`uart_puts`)
- Leer líneas completas (`uart_gets_input`)
- Inicializar el sistema

¿Por qué esta arquitectura importa?

- Cada capa se puede modificar de forma independiente. Por ejemplo, si queremos modificar lo que el usuario mirará, solo tenemos que modificar la capa de usuario, sin riesgo a arruinar más código.
- Los programas de usuario no pueden dañar directamente el hardware, todo debe pasar por la capa de sistema operativo.
- Podemos reutilizar las librerías para otros programas.
- Los errores son más fáciles de encontrar porque cada capa tiene una responsabilidad clara.

Pruebas

```
nataliasosa@Natalias-MacBook-Pro Files % ./build_and_run.sh

Cleaning up previous build files...
Assembling root.s...
Compiling user layer...
Compiling language library...
Compiling OS layer...
Linking object files...
Converting ELF to binary...
Running QEMU...
Program: Add Two Numbers
-----
[Enter first number: 544
[Enter second number: 123
544 + 123 = 667
[Enter first float number: 98.1
[Enter second float number: 11.9
98.09 + 11.89 = 110.00
-----
```

```
[nataliasosa@Natalias-MacBook-Pro Files % ./build_and_run.sh
[nataliasosa@Natalias-MacBook-Pro Files % ./build_and_run.sh
Cleaning up previous build files...
Assembling root.s...
Compiling user layer...
Compiling language library...
Compiling OS layer...
Linking object files...
Converting ELF to binary...
Running QEMU...
Program: Add Two Numbers
-----
[Enter first number: 342
[Enter second number: 1132
342 + 1132 = 1474
[Enter first float number: 9980.2
[Enter second float number: 112.1
998.00 + 112.09 = 1110.09
-----
Enter first number: █
```

Como algo extra, se agregó la posibilidad de poder borrar un carácter utilizando el backspace si se necesita modificar algún número.

```
//If backspace is pressed, handle it
if(c == '\b' || c == 127){
    if(i > 0){
        i--;
        uart_putc('\b'); //Move cursor back
        uart_putc(' '); //Erase character
        uart_putc('\b'); //Move back again
    }
    continue;
}
```